M.C.A. (Management)

JPR 551 MJ: Java Programming
(2024 Pattern) (Semester - II)

Time : 2½ Hours] ***[Max. Marks : 50]***

Instructions to the candidates:

1) ***All questions are compulsory.***
2) ***Figures to the right indicate full marks.***

Q1)Solve any two :

a) Define a class calculation to implement method overloading for addition of two integers and two double variables.

[5]

Answer:

Aim

- To create a Java class Calculation that demonstrates **method overloading** for adding two integers and two double values.

2. Theory

- **Method Overloading** is a feature in Java where multiple methods can have the **same name** but **different parameters** (type, number, or order).
- It improves code readability and flexibility.

3. Program

```
public class Calculation
{

   // Method to add two integers
   void add(int a, int b)
```

```
    {
        int c;
        c=a+b;
        System.out.println("Addition of integers: " +c);
    }
  // Method to add two double values
    void add(double a, double b)
    {
        double sum;
        sum=a+b;
        System.out.println("Addition of doubles: " +sum);
    }

    public static void main(String[] args)
    {

        Calculation obj = new Calculation();

        obj.add(10,20);       // calls int method
        obj.add(10.23,23.456);  // calls double method
    }
}
```

4. Output

Addition of integers: 30

Addition of doubles: 33.686

5. Explanation

- In this program, a class named Calculation is created with two methods having the same name add().
    - The first add() method takes **two integer parameters** and returns their sum.
    - The second add() method takes **two double parameters** and returns their sum.
- When the method is called using integer values, the integer version is executed.

When the method is called using double values, the double version is executed.

Thus, Java decides which method to call based on the **type of arguments passed**, demonstrating **method overloading**.

b) Define class person with suitable data member and methods. And extend this class in Manager class. Display manager details.

[5]

Answer:

1. Aim

- To create a base class Person and extend it into a derived class Manager using **inheritance**, and display manager details.

2. Theory

- **Inheritance** is a feature of Java in which one class (child class) acquires the properties and behaviors of another class (parent class).
- It promotes **code reusability** and establishes a relationship between classes.

3. Program

```
// Parent class
class Person
{
   int aadharID = 52341;
   String name = "Amit";
}

// Child class
class Manager extends Person
{
   int salary = 40000;
   int empid=101;

   void display()
   {
      System.out.println("Aadhar ID: " + aadharID);
      System.out.println("Name: " + name);
      System.out.println("Salary: " + salary);
```

```
        System.out.println("Employee Id: " + empid);
    }
}

// Main class
public class test
{
    public static void main(String[] args)
    {

        Manager m = new Manager();
        m.display();

    }
}
```

4. Output

Aadhar ID: 52341
Name: Amit
Salary: 40000
Employee Id: 101

5. Explanation

- This program demonstrates **inheritance** in Java.
- Person is the **parent class** containing aadharID and name.
- Manager is the **child class** that extends Person and adds salary and empid.
- The Manager class accesses and displays both **inherited** and **own data members** using the display() method.
- Thus, it shows **code reuse using inheritance**.

c) Write a function using lambda expression to calculate power of number.(x^y) [5]

Answer:

1. Aim

- To write a Java program using a **lambda expression** to calculate the power of a number.

2. Theory

- A **lambda expression** in Java is used to implement a **functional interface** (interface with one abstract method) in a short and simple way.
- It reduces code length and improves readability.

Syntax:

(argument list) -> {body of the expression}

- **Argument List:** Parameters for the lambda expression
- **Arrow Token (->):** Separates the parameter list and the body
- **Body:** Logic to be executed.

3. Program

```
interface Power
{
   double calculate(int x, int y);
}

public class Test
{
   public static void main(String[] args)
```

```
    {

        // Lambda expression
        Power p = (x, y) -> Math.pow(x, y);

        // Calling function
        System.out.println("Power is: " + p.calculate(2, 3));
    }
}
```

4. Output

Power is: 8.0

5. Explanation

1. Interface Power

Defines method calculate(x, y)

2. Lambda Expression

(x, y) -> Math.pow(x, y)

Takes two inputs and returns power

3. Math.pow()

Built-in method to calculate power

4. Calling method

p.calculate(2,3) → $2^3 = 8$

d) What is garbage collection? Explain with required method.

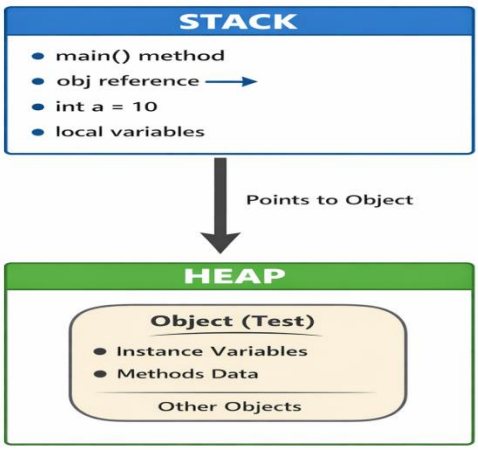
[5]

➢ What is Garbage Collector (GC)?

- Garbage Collector is a **memory management system** in Java that automatically removes unused objects from memory.
- It frees memory by deleting objects that are **no longer referenced** by any part of the program.
- This helps prevent **memory leaks** and improves performance.

Definition:

" Garbage Collection is a process that automatically destroys unused objects to free heap memory."



Real-Life Example

- Think of your computer desktop
- When files are not needed, you delete them to free space.
- Garbage Collector does the same with unused objects in memory.
-

Why Garbage Collection is Needed

- To free unused memory
- To avoid memory leaks
- To improve program performance
- To manage heap memory automatically
- No need for manual memory deletion

How Garbage Collector Works

- Java automatically checks objects in **Heap Memory**
- If an object has:

✕No reference
✕Not used anywhere

GC removes it and frees memory

Example of Garbage Collection

```
class Test
{
   public static void main(String[] args)
{

        Test t1 = new Test();
        Test t2 = new Test();

        t1 = null;  // eligible for GC
        t2 = null;  // eligible for GC

        System.gc(); // request GC
    }
}
```

Since objects have no reference → GC can remove them

Required Methods Related to Garbage Collection

1. System.gc()

- Requests JVM to run Garbage Collector
- It is NOT guaranteed to run immediately
- System.gc();

2. Runtime.getRuntime().gc()

- Another way to request GC

Runtime.getRuntime().gc();

3. finalize() Method

- Called by GC before destroying object
- Used to perform cleanup tasks

- **Example:**

```
class Test1
{

   protected void finalize()
   {
      System.out.println("Object destroyed");
   }

   public static void main(String[] args)
   {
      Test t = new Test();
      t = null;
      System.gc();
   }
}
```

Q2) Solve any two :

a) Create a thread to display prime numbers between 1 to 500 each number will display after 3 seconds. [5]

Answer:

1. Aim

- To create a thread in Java that prints **prime numbers from 1 to 500** with a delay of **3 seconds** between each number.

2. Theory

- A **thread** is a lightweight process used for executing tasks concurrently. In Java, a thread can be created by:
  - Extending Thread class
  - Implementing Runnable interface
  - sleep() method is used to pause execution for a specified time.

3. Program

```
class MyThread extends Thread
{
    public void run()
    {
        for(int i=2;i<=500;i++)
        {
            int flag=0;

            for(int j=2;j<i;j++)
            {
```

```
                if(i%j==0)
                {
                    flag=1;
                    break;
                }
            }

            if(flag==0)
            {
                System.out.println(i);

                try{
                    Thread.sleep(3000);   // 3 seconds delay
                }
                catch(Exception e){}
            }
        }
    }
}

class DemoPrime
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

4. Output

2
3
5
7
11
13
17
19
23
29
…

(Each number prints after 3 seconds)

5. Explanation

- This program creates a **thread by extending the Thread class**.
- The run() method checks numbers from **2 to 500** to find **prime numbers**.
- If a number is prime, it is printed using System.out.println().
- After printing each prime number, the thread pauses for **3 seconds using** Thread.sleep(3000).
- The thread starts execution using the start() **method** in the main() function.

b) Write a java program to demonstrate how to create user-defined exception. [5]

Answer:

- Java allows programmers to **create their own custom exceptions**.
- These are called **User Defined Exceptions**.
- A user-defined exception is created by **extending the Exception class**.

Steps to Create User Defined Exception

1. Create a class that **extends Exception**.
2. Create a **constructor** to display the error message.
3. Use the **throw keyword** to throw the exception.
4. Handle it using **try-catch block**.

Syntax

```
class MyException extends Exception
{
   MyException(String message)
   {
      super(message);
   }
}
```

super(message) is used to pass the message to the Exception class.

Example

```
class MyException extends Exception
{
    MyException(String msg)
    {
        super(msg);
    }
}

class Test6
{
    public static void main(String args[])
    {
        try
        {
```

```
            throw new MyException("This is custom exception");
        }
        catch(MyException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

This is custom exception

Explanation:

1. MyException is a **user defined exception class** that extends the Exception class.
2. The constructor passes the **error message** using super(msg).
3. In main(), the program **throws the custom exception** using throw.
4. The catch block **handles the exception** and prints the message using e.getMessage().

c) Write a Java program to implement arraylist with function to add, remove & sort member. [5]

Answer:

1. Aim:

- To create an ArrayList and perform operations like adding, removing, and sorting elements.

2. Theory:

- ArrayList is a class in the Java Collection Framework available in java.util package.
- It is a **dynamic array**, which means its size can increase or decrease automatically.
- It maintains **insertion order** and allows **duplicate elements**.
- Common methods used:
  - add() → adds elements to the list
  - remove() → removes elements from the list
  - Collections.sort() → sorts elements in ascending order

3. Program:

```
import java.util.ArrayList;
import java.util.Collections;
class ArrayListDemo
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();
        // Add elements
        list.add("Sahil");
        list.add("Rahul");
```

```
        list.add("Amit");
        list.add("Neha");
        System.out.println("ArrayList after adding elements:");
        System.out.println(list);

        // Remove element
        list.remove("Rahul");

        System.out.println("ArrayList after removing element:");
        System.out.println(list);

        // Sort elements
        Collections.sort(list);

        System.out.println("ArrayList after sorting:");
        System.out.println(list);
    }
}
```

4. Output:

ArrayList after adding elements:
[Sahil, Rahul, Amit, Neha]
ArrayList after removing element:
[Sahil, Amit, Neha]

ArrayList after sorting:
[Amit, Neha, Sahil]

5. Explanation:

1. ArrayList<String> list = new ArrayList<>(); → Creates a dynamic list of strings.
2. list.add() → Adds elements to the list.
3. list.remove() → Removes an element from the list.
4. Collections.sort() → Sorts the elements in ascending order.

The program shows how to **add, remove, and sort elements in an ArrayList**.

d) Differentiate between checked and unchecked exception [5]

Answer:

No.	Checked Exception	Unchecked Exception
1	Checked exceptions are checked by the compiler at **compile time**.	Unchecked exceptions are checked at **runtime**.
2	Checked exceptions **must be handled** using try-catch or throws.	Unchecked exceptions **do not require mandatory handling**.
3	Checked exceptions usually occur due to **external problems** like file or database issues.	Unchecked exceptions usually occur due to **programming mistakes**.
4	If a checked exception is not handled, the **program will not compile**.	If an unchecked exception is not handled, the **program compiles but fails at runtime**.
5	Checked exceptions are subclasses of Exception (except runtime exceptions).	Unchecked exceptions are subclasses of RuntimeException.
6	Checked exceptions help in **improving program reliability** because they must be handled.	Unchecked exceptions usually indicate **logical errors in the program**.
7	Checked exceptions commonly occur in **input-output operations**.	Unchecked exceptions occur during **incorrect calculations or null references**.
8	Example: IOException occurs during file handling.	Example: ArithmeticException occurs when dividing by zero.
9	Another example of checked exception is SQLException.	Another example of unchecked exception is NullPointerException.
10	These exceptions are **checked by the compiler before program execution**.	These exceptions are **identified only when the program runs**.

Q3) Solve any one :

a) Create a HTML page to accept two numbers and write servlet to add given numbers and display result. [10]

Answer:

Project Setup in Eclipse

Step 1: Create Dynamic Web Project

1. Open Eclipse
2. Go to **File → New → Dynamic Web Project**
3. Project Name: AdditionServletApp
4. Target Runtime: Select Apache Tomcat (configure if not added)
5. Click **Finish**

Step 2: Create HTML File

1. Right click project → **New → HTML File**
2. Name: index.html

✨HTML Code[index.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Addition Form</title>
</head>

<body>
<h2>Add Two Numbers</h2>

<form action="add" method="post">
Enter Number 1: <input type="text" name="num1"><br><br>
Enter Number 2: <input type="text" name="num2"><br><br>
<input type="submit" value="Add">
</form>
</body>
</html>
```

Step 3: Create Servlet Class

1. Right click project → **New** → **Servlet**
2. Class Name: AddServlet
3. Package: com.example
4. Click **Finish**

✨Servlet Code (AddServlet.java)

```
package com.example;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/add")
public class AddServlet extends HttpServlet
{

protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

int num1 = Integer.parseInt(request.getParameter("num1"));
int num2 = Integer.parseInt(request.getParameter("num2"));

int result = num1 + num2;

out.println("<h2>Result: " + result + "</h2>");
}
}
```

4. Configure Server (Tomcat)

1. Go to **Servers tab**
2. Click **"No servers available" → Add new server**
3. Select **Apache Tomcat**
4. Browse and select Tomcat installation folder
5. Click **Finish**

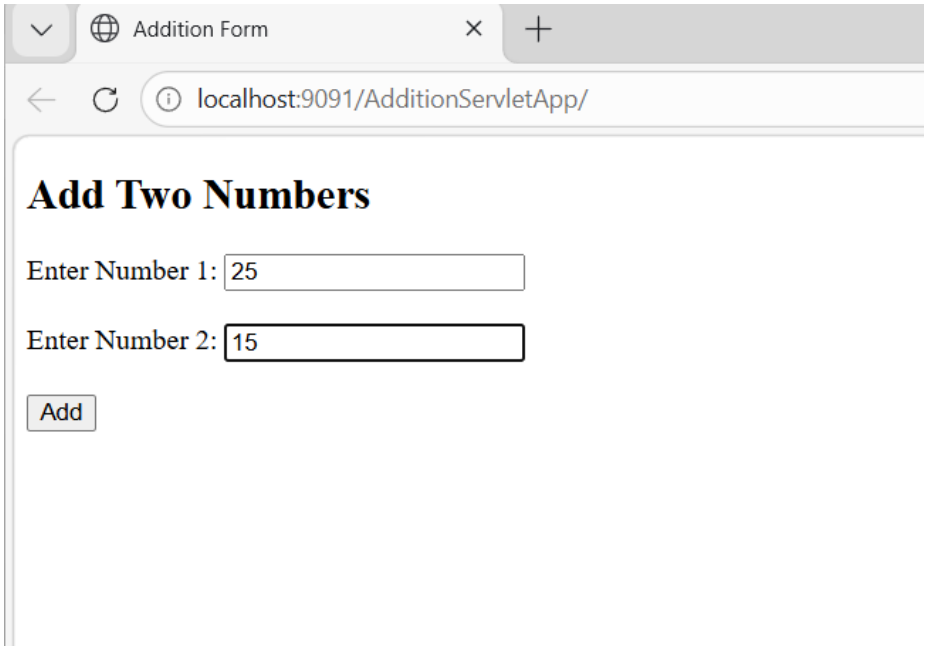
5. Run the Project

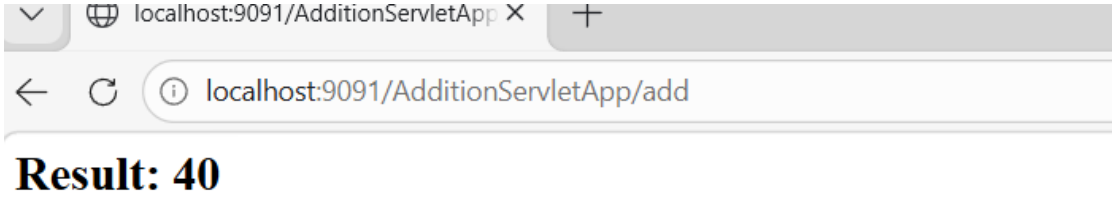
1. Right click project → **Run As → Run on Server**
2. Choose Tomcat → Finish

6. Output

Open browser:

http://localhost:8080/AdditionServletApp/index.html





Key Points

- HTML form sends data using **POST method**
- request.getParameter() is used to read input
- Integer.parseInt() converts string to int
- Servlet processes logic and returns response using PrintWriter

b) Explain servlet life cycle and demonstrate its methods with example.[10]

Introduction to Servlet Life Cycle

- The **Servlet Life Cycle** defines the stages through which a servlet passes from its **creation to destruction**.
- It is **controlled by the web server** like Apache Tomcat.

2. Phases of Servlet Life Cycle

- A servlet has **three main life cycle methods**:

Phase 1: Loading and Instantiation

- The servlet class is **loaded into memory** by the server
- An **object of the servlet is created**
- This happens **only once**

Loading can be:

- Automatically (first request)
- At server startup

Phase 2: Initialization – init() Method

✓ Definition:

The init() method is called **only once** after the servlet object is created.

✓ Syntax:

public void init() throws ServletException

✓ Purpose:

- Initialize resources
- Setup configuration
- Establish database connection

✓ Key Points:

- Called only once in lifetime
- Cannot be called again and again

Phase 3: Request Processing – service() Method

✓ Definition:

The service() method is called **every time a client sends a request**.

✓ Syntax:

public void service(ServletRequest req, ServletResponse res)

✓ **Working:**

- It is the **heart of servlet processing**
- It decides which method to execute:
    - doGet() → for GET request
    - doPost() → for POST request

In real applications, we override:

- doGet() or doPost() instead of service()

Phase 4: Destruction – destroy() Method

✓ **Definition:**

The destroy() method is called **once before the servlet is removed from memory**.

✓ **Syntax:**

public void destroy()

✓ **Purpose:**

- Release resources
- Close database connections
- Clean up memory

Life Cycle Diagram:

Detailed Flow

1. Servlet is loaded by server (Apache tomcat)
2. init() is called (once)
3. For each request:
    - service() executes
    - Calls doGet() / doPost()
4. When server stops:
    - destroy() is called

Example Program Demonstrating Life Cycle

```
package com.demo;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LifeCycleServlet")
public class LifeCycleServlet extends HttpServlet
{

public void init()
{
```

```
System.out.println("Servlet Initialized");
}

protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws IOException
{

response.setContentType("text/html");
PrintWriter out = response.getWriter();

out.println("<h2>Service Method Executed</h2>");
out.close();
}

public void destroy()
{
System.out.println("Servlet Destroyed");
}
}
```

Output:

Service Method Executed

Q4) Solve any one :

a) Design Java Application (using JSP) for registration of hackthon (student name, Mail Id, Phone No, Gender, Course) and store data in appropriate table. [10]

Answer:

1. Aim

To develop a **JSP-based Hackathon Registration System** and store student details in MySQL database.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Creation (Using test DB)

Step 1: Open MySQL Workbench

Step 2: Use Database

```
USE test;
```

Step 3: Create Table

```
CREATE TABLE hackathon
(
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   email VARCHAR(100),
   phone VARCHAR(15),
   gender VARCHAR(10),
   course VARCHAR(50)
);
```

4. Project Creation in Eclipse

Steps:

1. Open Eclipse
2. File → New → Dynamic Web Project
3. Project Name: HackathonRegistration
4. Select Apache Tomcat
5. Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

1. Right click your project
2. Click:
   Build Path → Configure Build Path
3. Go to:
   Libraries tab - select ClassPath
4. Click:
   Add External JARs
5. Select your file:

mysql-connector-j-9.6.0.jar

Click:
Apply → OK

✓ Now check:
Referenced Libraries
➞You should see your JAR file:mysql-connector-j-9.6.0.jar

6.JSP Form Page

STEP : Create 2 JSP Files

register.jsp
save.jsp

Right click project → **New** → **JSP File**

register.jsp

```
<!DOCTYPE html>
<html>
<body>
<h2>Hackathon Registration</h2>

<form action="save.jsp" method="post">
Name: <input type="text" name="name"><br><br>
Email: <input type="email" name="email"><br><br>
Phone: <input type="text" name="phone"><br><br>

Gender:
<input type="radio" name="gender" value="Male">Male
<input type="radio" name="gender" value="Female">Female<br><br>

Course:
<select name="course">
<option>MCA</option>
<option>BCA</option>
<option>BBA</option>
</select><br><br>

<input type="submit" value="Register">
</form>

</body>
</html>
```

7. JSP Database Code (IMPORTANT CHANGE)

save.jsp

```
<%@ page import="java.sql.*" %>

<%
String name = request.getParameter("name");
String email = request.getParameter("email");
String phone = request.getParameter("phone");
String gender = request.getParameter("gender");
String course = request.getParameter("course");

try
{
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test?useSSL=false&serverTimezone=UTC",
"root",
"root"
);

PreparedStatement ps = con.prepareStatement(
"INSERT INTO hackathon(name,email,phone,gender,course)
VALUES(?,?,?,?,?)"
);
ps.setString(1, name);
ps.setString(2, email);
ps.setString(3, phone);
ps.setString(4, gender);
ps.setString(5, course);

ps.executeUpdate();

out.println("<h2>Registration Successful!</h2>");

con.close();

} catch(Exception e) {
out.println(e);
}
%>
```

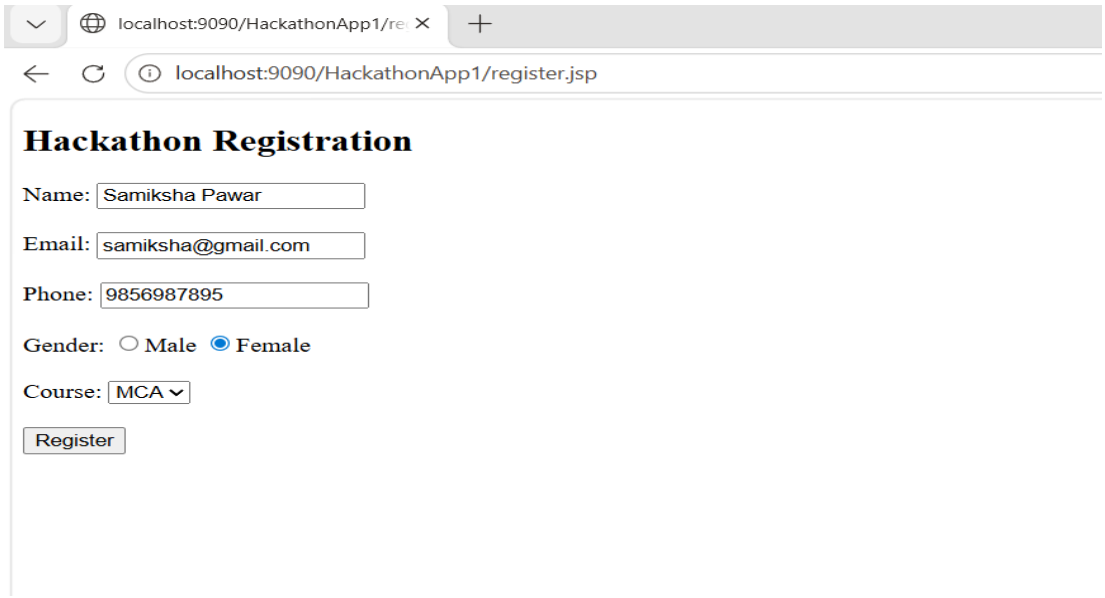
8. Execution Steps

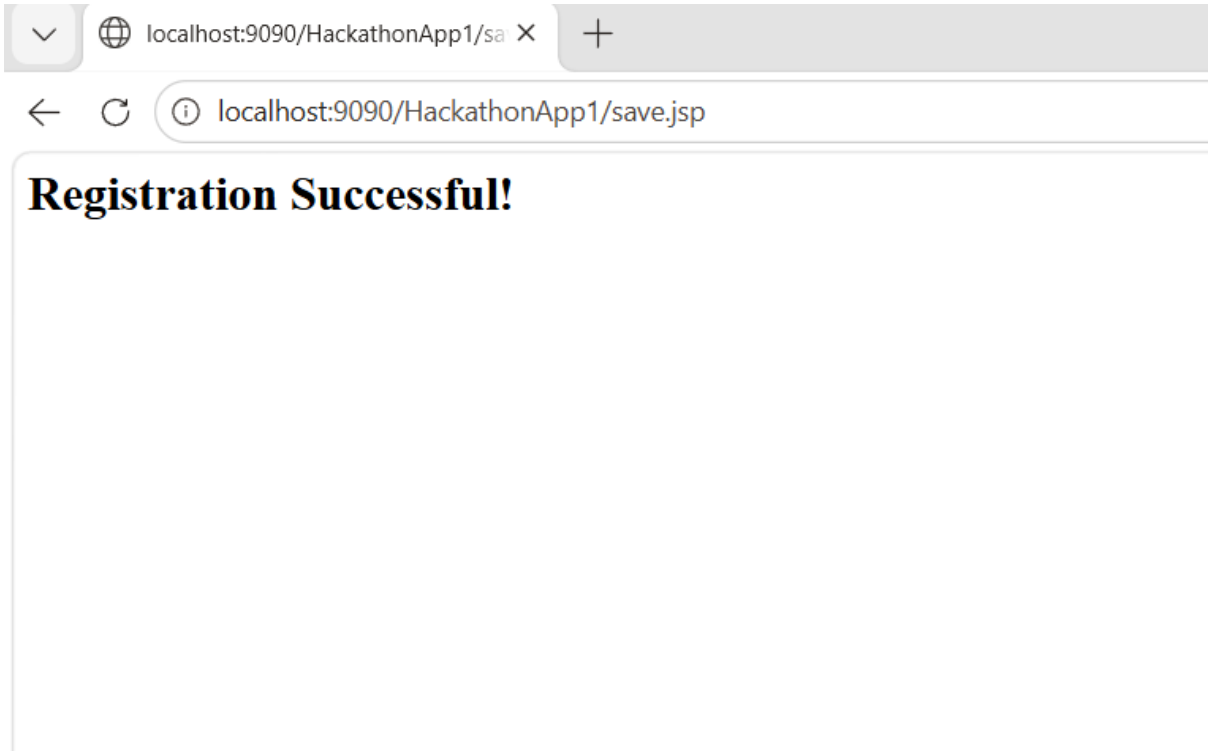
1. Right click project
2. Run on Server
3. Select Apache Tomcat

Open:

http://localhost:8080/HackathonRegistration/register.jsp

9. Output



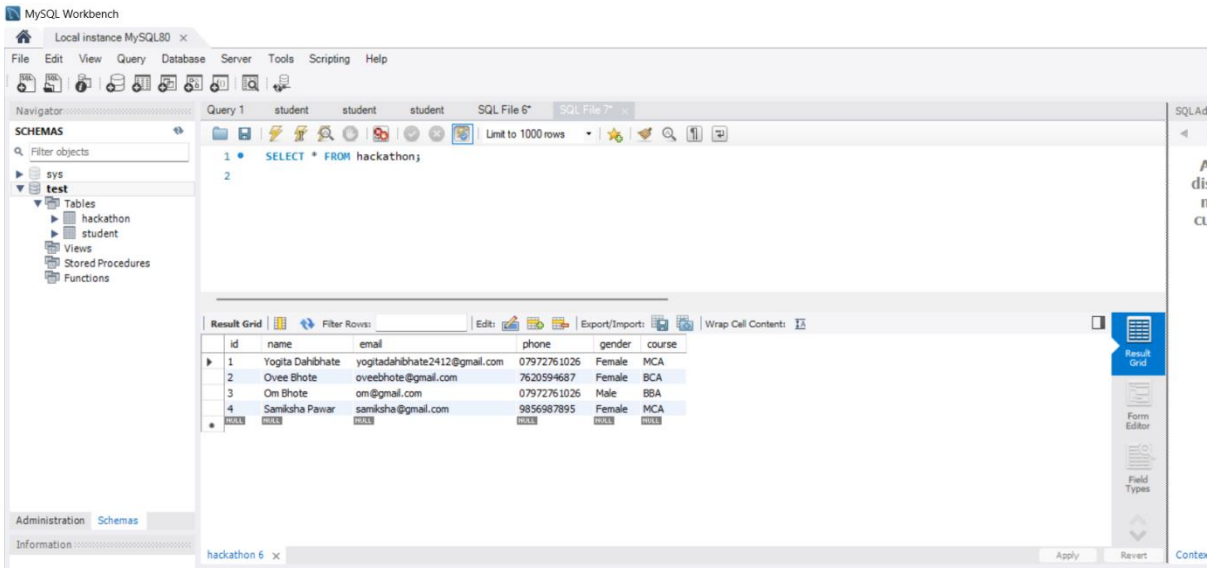


9. Verify Data in Database

Open MySQL Workbench

Run:

SELECT * FROM hackathon;



b) Explain JSP directives with example. [10]

1. JSP Directives

- Used to provide instructions to JSP container.
- **JSP Directives** are special instructions given to the JSP container (server like Apache Tomcat) to control how the JSP page is processed.
- They **do not produce output directly**, but they affect the **overall structure and behavior** of the JSP page.

✓ Syntax:

<%@ directive attribute="value" %>

Types of JSP Directives:

a) Page Directive

- The **page directive** is used to define **page-level settings** such as language, content type, import packages, error handling, etc.
- It defines page-level settings.
- Syntax:

<%@ page language="java" contentType="text/html" %>

Common Attributes of Page Directives

Attribute	Description
language	Specifies programming language (Java)
contentType	Defines MIME type
import	Imports Java packages
errorPage	Defines error page
isErrorPage	Specifies error handling page

b) Include Directive

- The **include directive** is used to include another file **at compile time**.
- It is useful for **header, footer, menu reuse**
- **Syntax:**

<%@ include file="header.jsp" %>

c) Taglib Directive

- The **taglib directive** is used to use **custom tags (libraries)** in JSP.
- Syntax:
  <%@ taglib uri="tag-library-uri" prefix="t" %>

Example 2: JSP Directives

STEP 1: Create Project

1. Open Eclipse
2. Go to:File → New → Dynamic Web Project
3. Project Name:JSPDirectiveDemo
4. Select **Apache Tomcat**
5. Click **Finish**

STEP 2: Create JSP Files

Create 3 files:

index.jsp
header.jsp
error.jsp

Right click project → **New** → **JSP File**

1. PAGE DIRECTIVE EXAMPLE

index.jsp

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" errorPage="error.jsp" %>
<%@ include file="header.jsp" %>
```

```
<!DOCTYPE html>
<html>
<head>
<title>Main JSP Page</title>
</head>
<body>

<h2>Main Page (index.jsp)</h2>

<p>This content is from index.jsp</p>

<hr>

<h3>Now generating an error intentionally:</h3>

<%
int a = 10;
int b = 0;
int c = a / b; // This will cause error
%>

</body>
</html>
```

error.jsp

```
<%@ page isErrorPage="true" %>

<!DOCTYPE html>
<html>
<head>
<title>Error Page</title>
</head>
<body>
<h2 style="color:red;">Error Occurred!</h2>

<p>Sorry, something went wrong.</p>

</body>
</html>
```

2. INCLUDE DIRECTIVE EXAMPLE

header.jsp

```
<!DOCTYPE html>

<html>
<body>

<h1 style="color:blue;">Welcome to JSP Directive Demo</h1>
<hr>

</body>
</html>
```

STEP 3: Run Project

1. Right click project
2. Click:Run As → Run on Server
3. Open browser:

http://localhost:8080/JSPAllDirectiveDemo/index.jsp

OUTPUTS

✓ Page Directive (Error Handling)

Error Occurred!
Sorry, something went wrong.

✓ Include Directive

Welcome Header
Main Page Content

Q5) Solve any one :

a) Create a spring MVC form to read registration details for Blood donation camp with spring validation & display it. [10]

Answer:

STEP 1: Create Spring Boot Project

1. Open IntelliJ
2. Click **New Project**
3. Select **Spring Boot**
4. Fill:
    - Name: bloodcamp
    - Language: Java
    - Packaging: Jar
5. Add Dependencies:
    - ✓Spring Web
    - ✓Thymeleaf
    - ✓Validation

Click **Finish**

STEP 2: Project Structure (Auto Created)

You will see:

src/main/java
src/main/resources

STEP 3: Create Model Class

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New → Package**
- Name it: model

2. Create Java Class

- Right click on model
- Click **New → Java Class**
- Name: Donor

Code: Donor.java

```
package com.blood.bloodcamp.model;

import jakarta.validation.constraints.*;

public class Donor
{

    @NotEmpty(message="Name is required")
    private String name;

    @Email(message="Invalid email")
    private String email;

    @Min(value=18, message="Minimum age is 18")
    private int age;

    @NotEmpty(message="Blood group required")
    private String bloodGroup;

    // Getters and Setters
    public String getName()
    { return name; }
    public void setName(String name)
    { this.name = name; }

    public String getEmail()
     { return email; }
    public void setEmail(String email)
    { this.email = email; }

    public int getAge()
    { return age; }
    public void setAge(int age)
```

```
    { this.age = age; }

    public String getBloodGroup()
    { return bloodGroup; }
    public void setBloodGroup(String bloodGroup)
    { this.bloodGroup = bloodGroup; }
}
```

STEP 4: Create Controller

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New → Package**
- Name it: controller

2. Create Java Class

- Right click on model
- Click **New → Java Class**
- Name: DonorController

Code: DonorController.java

```
package com.blood.bloodcamp.controller;

import com.blood.bloodcamp.model.Donor;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class DonorController {

    @GetMapping("/")
    public String showForm(Model model) {
        model.addAttribute("donor", new Donor());
        return "form";
```

```
    }

    @PostMapping("/submit")
    public String submitForm(@Valid @ModelAttribute("donor") Donor donor,
                             BindingResult result) {

        if(result.hasErrors()) {
            return "form";
        }

        return "result";
    }
}
```

STEP 5: Create HTML Pages

Path: src/main/resources/templates

Code: form.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Blood Donation Form</h2>

<form th:action="@{/submit}" th:object="${donor}" method="post">

    Name: <input type="text" th:field="*{name}"/>
    <span th:errors="*{name}"></span><br><br>

    Email: <input type="text" th:field="*{email}"/>
    <span th:errors="*{email}"></span><br><br>

    Age: <input type="text" th:field="*{age}"/>
    <span th:errors="*{age}"></span><br><br>

    Blood Group: <input type="text" th:field="*{bloodGroup}"/>
    <span th:errors="*{bloodGroup}"></span><br><br>

    <button type="submit">Submit</button>
```

```
</form>

</body>
</html>
```

Code: result.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Submitted Details</h2>

Name: <span th:text="${donor.name}"></span><br>
Email: <span th:text="${donor.email}"></span><br>
Age: <span th:text="${donor.age}"></span><br>
Blood Group: <span th:text="${donor.bloodGroup}"></span><br>

</body>
</html>
```

STEP 6: Run Project

1. Open main file: BloodcampApplication.java
2. Click **Run** ▶

STEP 7: Open in Browser

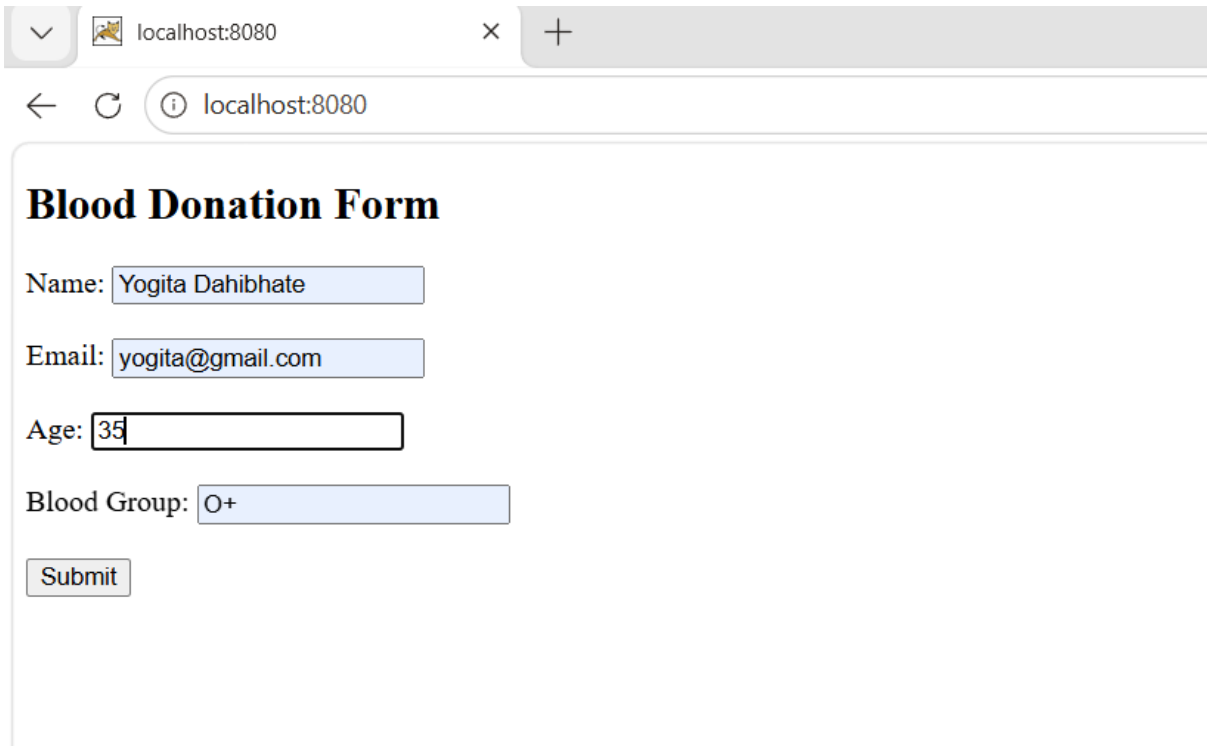
Go to:

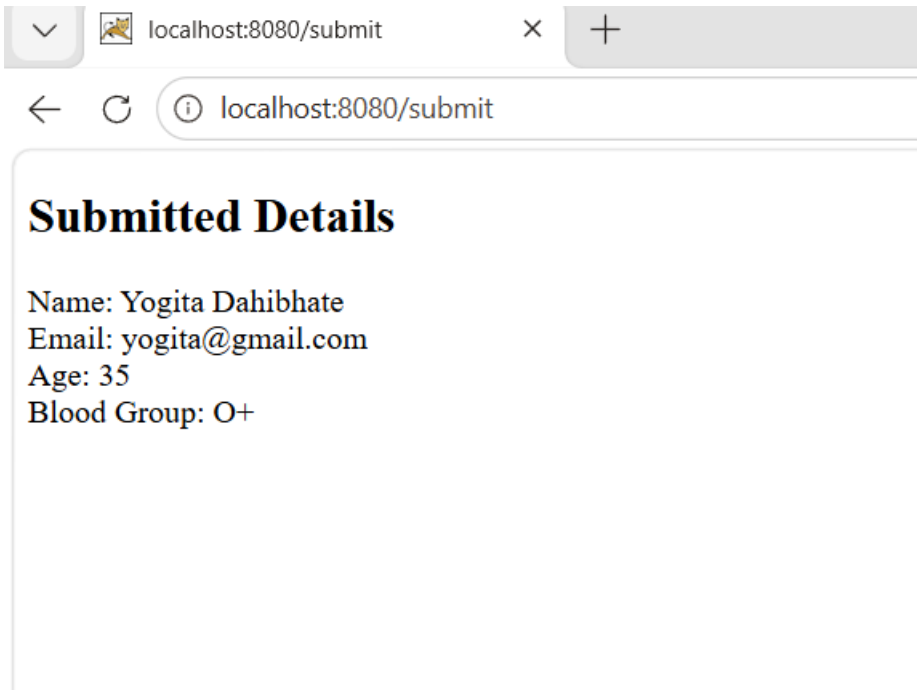
http://localhost:8080/

OUTPUT

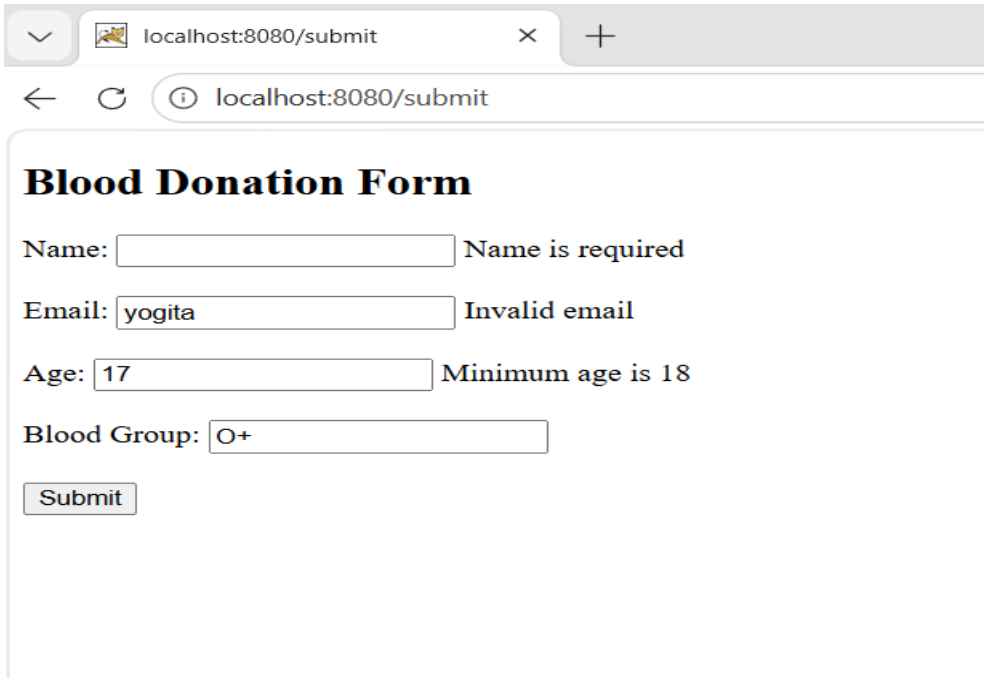
- Form will open
- Enter data

- Validation works
- Data displayed on next page





Validation:



b) Explain spring MVC architecture. And also explain any two spring annotations. [10]

Answer:

Introduction spring MVC architecture.

- Spring MVC is a **Java web framework** used to develop **web applications** based on the **Model–View–Controller (MVC)** pattern.
- It separates application into:
  - **Model** → Data
  - **View** → Presentation (UI)
  - **Controller** → Business logic
- This separation makes the application:
  - Easy to understand
  - Easy to maintain
  - Scalable

2. Core Components of Spring MVC:

1. DispatcherServlet (Front Controller)

- Central controller of Spring MVC
- Receives **all incoming requests**
- Coordinates entire request flow

Responsibilities:

- Send request to handler mapping
- Call appropriate controller
- Send response to view

2. Handler Mapping

- Maps **URL → Controller**
- Helps DispatcherServlet find correct controller

Example:

/home → HomeController

3. Handler Adapter

- Acts as a bridge between **DispatcherServlet and Controller**
- Calls controller method properly

Needed because controllers can be of different types

4. Controller

- Processes request
- Contains business logic
- Returns:
    - Model data
    - Logical view name

5. Model

- Holds data (like student details, form data)
- Passed from controller to view

Example:

model.addAttribute("name", "Yogita");

6. View Resolver

- Converts logical view name → actual view file

Example:

"home" → /WEB-INF/views/home.jsp

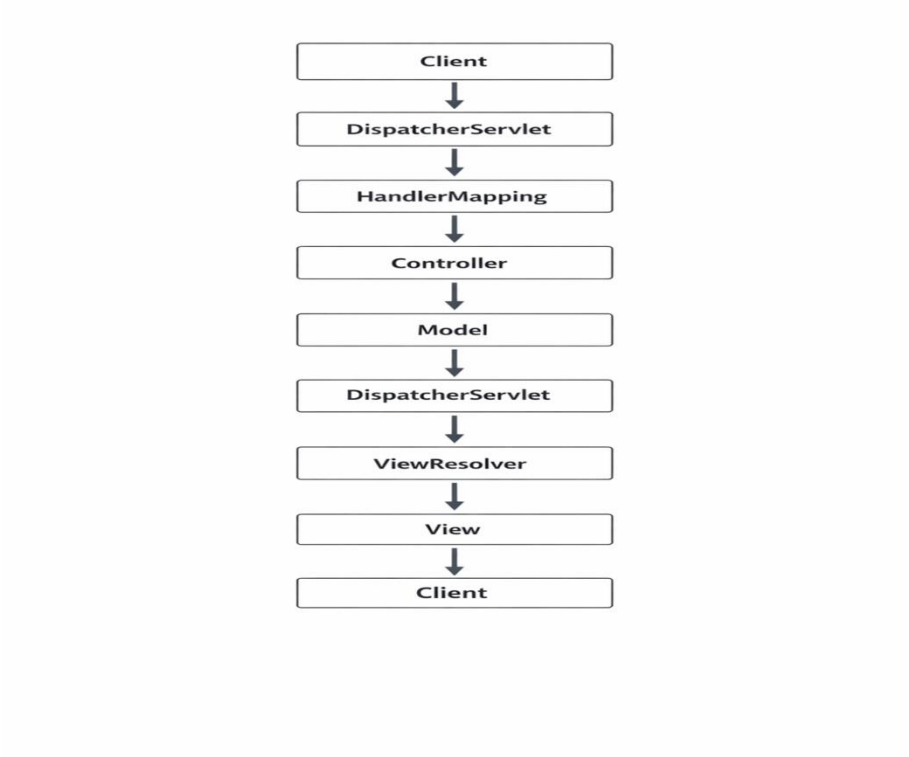
7. View

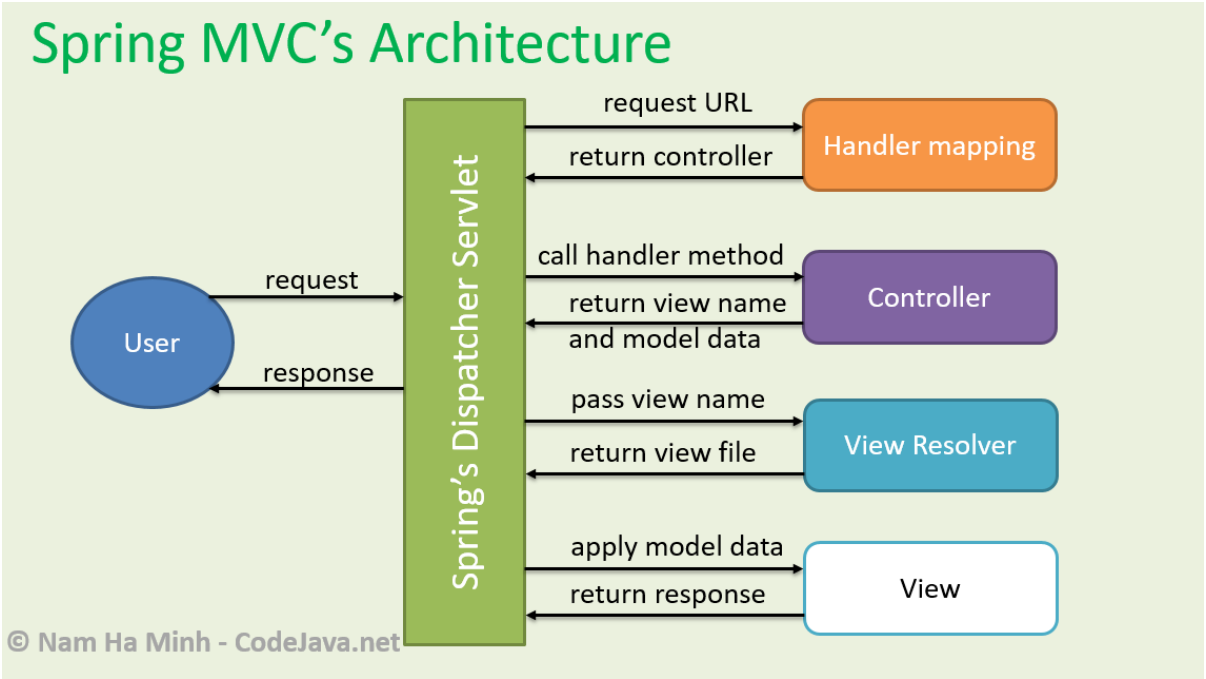
- Final output shown to user
- Technologies:
    - JSP
    - HTML
    - Thymeleaf

3. Complete Working Flow (Step-by-Step):

1. User sends HTTP request (browser)
2. Request reaches **DispatcherServlet**
3. DispatcherServlet asks **Handler Mapping** for controller
4. Handler Mapping returns controller info
5. DispatcherServlet uses **Handler Adapter**
6. Controller method is executed
7. Controller processes logic
8. Data is stored in **Model**
9. Controller returns logical view name
10. DispatcherServlet sends it to **View Resolver**
11. View Resolver finds actual view (JSP/HTML)
12. View displays response to user

4. Diagram Explanation





5. Advantages of Spring MVC

- Clear separation of concerns
- Reusable components
- Easy testing
- Flexible view technologies
- Scalable architecture

6. Real-Life Example

Online Shopping Website

- User clicks product → Request sent
- Controller fetches product data
- Model stores data
- View displays product page